This problem set is not going to be collected, although solutions will be posted before the quiz. I recommend you treat this as if this were due Tuesday of week 5 at lecture. In addition to solving them, I recommend writing up solutions as if you were going to submit them. Unlike a problem set collected for credit, though, you are free to discuss with classmates and course staff and get help.

I recommend you first attempt the problems on your own, then discuss with classmates. After that, you may want to consult course staff at help hours or on Ed Discussion. After solutions are posted, consult the solutions and compare to what you wrote, and then discuss the differences with your study group.

All questions after the first on this problem set, and also one question on your first topic exam, will require a dynamic programming solution. For any such problem, you **must** do the following; missing one of these can cost you significant credit.

- Give a clear and precise English definition that describes the function you are implementing. Not *how* it works (yet), but rather *what* it solves.
- Give that function a meaningful variable name. This is not [just] me being pedantic; I have found it helps students with this topic if they do this. "OPT" is not a meaningful variable name, nor is "dp" nor "table." Single letters are not meaningful variable names. Yes, I know you're going to read question one first, and yes, I read it too.
- Give a clear recursive formula or algorithm in terms of smaller instances of *exactly* the same problem. If you aren't solving exactly the same problem, you might need to go back to the previous step.
- Describe the iterative running time correctly. This can either be by writing the iterative algorithm (in which case, you can point out where the previous part is within the solution), or by taking your recursive solution, counting the cases, describing the order in which the table would be filled in, and analyzing the time accordingly.

1. Suppose I am going to choose an integer between 1 and n, inclusive, according to some probability distribution. For each integer i, I have written p_i, the probability that I select i as the chosen integer. You may assume that $\sum_{i=1}^{n} p_i = 1$.

   (a) Give an $\mathcal{O}(n^3)$ time algorithm to compute a $2D$-array X, where $X[i, j]$ is the probability that some integer in the range $[i, j]$ (inclusive) is chosen. You may assume that arithmetic operations take $\mathcal{O}(1)$ time each.

   (b) Give an $\mathcal{O}(n^2)$ time algorithm to solve the problem in part (a). If you are confident that your answer to this question is $\mathcal{O}(n^2)$, you may elect to skip the previous part and count this as your answer to both.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

2. *Skittles* are small candies; each piece of candy is one of five flavors (grape, lemon, orange, strawberry, or green). An exciting new party game, *Skittle Trader*, is sweeping the nation, and the skills you are learning in CompSci 161 will help you to achieve a high score!

   Here are the rules of the game. You are at a party with n of your friends. Each of you has one piece of *Skittles* candy. Your friends form a line. You walk down the line, starting at friend 1 and finishing at friend n. At each friend, you have a choice: do you want to trade the candy you are holding for the one they are holding?

   - If you do not trade candy, your score remains the same.
   - If you and the friend have the same flavor candy, you earn one point.
   - If you and the friend trade, but you had different flavor types, you lose one point. Your score *can* go negative.

   You begin the game with zero points. Your goal is to finish the game with the most possible points.

   For purposes of the input to this problem, the Skittles flavors are numbers $0, 1, 2, 3, 4$. Your input to this problem is an array $C[1 \ldots n]$, where each $C[i] \in [0, 4]$ is the type of candy the ith friend in line is holding. You also have a parameter for the type of candy you begin the game with.

   If you decide to actually play this game with friends, perhaps to test out your algorithm before submitting, please use individually wrapped candies instead, such as Starburst. This is nearly the most disgusting question I have given in a class, second only to the Chicken McNuggets question from ICS 46.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

3. You've decided to play some music late night at your local club and want to impress. To prepare your strategy, you have reviewed your notes and used statistical analysis to figure out which moves get the most applause ending at certain musical measures. In your findings you've found that at each measure, you can perform one of three different moves.

   - **Jazzy riff:** *This is short and improvised.*
     How long it lasts is constant, but how you play depends on the ending measure.
     Played for 1 measure until the end of measure t, it awards you j_t applause. Note that j is a vector: the measure on which you end the riff affects the applause earned.
   - **Instrumental chorus:** *Your bread and butter.*
     Both how long it lasts and how you play is constant.
     Played for 2 measures until the end of measure t, it awards you C applause. Note that C is a constant: an instrumental chorus always earned you C applause.
   - **Solo:** *You play your heart out!*
     Both how long it lasts and how you play depends on the ending measure.
     Played for k_t measures until the end of measure t, it awards you s_t applause. Note that k and s are vectors: the measure on which you end a solo affects the applause earned.

   Given these statistical models (the vectors j, k, s and the constant C) are correct, you want to figure out **the maximum amount of applause** you can obtain in n musical measures. *Solutions that omit the inclusion of the "solo" option but are otherwise correct will receive reasonable partial credit.*

4. When their respective sport is not in season, UCI's student-athletes are very involved in their community, helping people and spreading goodwill for the school. Unfortunately, NCAA[1] regulations limit each student-athlete to at most one community service project per semester, so the athletic department is not always able to help every deserving charity. For the upcoming quarter, we have S student-athletes who want to volunteer their time, and B buses to help get them between campus and the location of their volunteering[2]. There are F projects under consideration; project i requires s_i student-athletes and b_i buses to accomplish, and will generate $g_i > 0$ units of goodwill for the university.

   If you are having trouble solving this problem, look at the **0-1 Knapsack Problem** in the textbook.

[1] Regulations mentioned in this problem are not necessarily accurate to reality and should be considered parody.
[2] The NCAA also won't permit student-athletes to use their own vehicles for these purposes. Furthermore, each bus can only be used for one project.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

5. Planning lectures can be quite the challenge. Luckily, I have already made a few key decisions already.

   - Use at most M minutes per lecture.
   - Teach all n topics and teach them in order (Topic 1 must be taught before Topic 3).
   - Allocate topic i exactly t_i minutes.
   - Topics cannot run over into another lecture (It is guaranteed that $\forall i\ t_i \leq M$).
   - I can have as many lectures as I want.

   If I was trying to minimize the number of lectures, I could just try to cover as many topics as possible per lecture until I finish them. However, my goal is to provide the best education experience and I have noticed that some topics work together better than others. According to my findings, I have found students receive E_{ij} engagement when I cover topics i through j in the same lecture given that $i \leq j$.

   If you would like to assume that $E_{ij} = -\infty$ whenever topics $i \ldots j$ do not fit into a single lecture spot, you may.

   Given the topic time list t and the engagement matrix E, design a dynamic programming solution to find the maximum total engagement I can achieve by teaching all n topics in order without exceeding M minutes per lecture.

6. We are given an array $A[1 \ldots n]$ for some odd n. Each odd-indexed location contains a positive number, and every even-indexed location contains either a $\times$ or a $+$. This forms an expression of n positive numbers, separated by multiplies and adds, but with the parenthesis removed so it is not clear what the order of operations was intended to be. Our goal is to determine the maximum possible result attainable by adding parenthesis. For example if the expression is: $2 * 15 + 4 * 0.5$ then we would output 34, which is attainable by $2 * (15 + (4 * 0.5))$ (although we only need to return 34, not the expression).

Recommended Book Problems

If you want additional problems to try, consider trying R-12.7, R-12.8, C-12.1, C-12.9, C-12.16, A-12.1, A-12.2, A-12.3, A-12.4, A-12.6, A-12.10 in the textbook of Goodrich and Tamassia or Chapter 3, problems 1, 2, 3, 6, 9, 11, 12, 13, 16, 17, 18, 23, 32, 33, 35, 46, or 49 in the textbook of Erickson. For a big challenge, try question 28 in that chapter.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.